

Defect detection in metal objects using a pre-trained Vision Transformer model.

Vincent Engeln
STUDENT NUMBER: 2051553
WORD COUNT: 8188

THESIS SUBMITTED IN PARTIAL FULFILLMENT
OF THE REQUIREMENTS FOR THE DEGREE OF
MASTER OF SCIENCE IN DATA SCIENCE & SOCIETY
DEPARTMENT OF COGNITIVE SCIENCE & ARTIFICIAL INTELLIGENCE
SCHOOL OF HUMANITIES AND DIGITAL SCIENCES
TILBURG UNIVERSITY

Thesis committee:

dr. Yash Satsangi
dr. Merel Jung
dr. Gökem Saygili

Tilburg University
School of Humanities and Digital Sciences
Department of Cognitive Science & Artificial Intelligence
Tilburg, The Netherlands
January 2022

Defect detection in metal objects using a pre-trained Vision Transformer model.

Vincent Engeln

Within industry, manufacturers make use of image classification algorithms for classifying products as defective or as non-defective. This method is called Automated Optical Inspection (AOI), and the Convolutional Neural Network (CNN) is the preferred model for this type of applications (Abu Ebayyeh 2020). However, there is a novel approach in the form of the Vision Transformer (ViT), which has shown to outperform state of the art CNN models while requiring far fewer computational resources (factor of 2 to 4) (Dosovitskiy et al. 2021). The ViT has proved to be successful in detecting defects in steel plates (Fan 2021), and accurately classifying building materials from a combined construction dataset (Soleymani et al. 2021). However, little research has been done into if a ViT can be considered as a more generic classification algorithm that can be considered for AOI applications. This leads us to the primary research question of this thesis: "Can a Vision Transformer serve as a universal metal defect classification model." In order to answer this question, three publicly available defect classification datasets containing metal casted products, metal pistons, and metal plates are combined into one dataset for the ViT to be tested on. In order to mitigate the class imbalance of the dataset several techniques were researched. Using a pre-trained ViT model, "Synthetic Minority Oversampling Technique" (SMOTE), and combining two similar classes resulted in the best prediction results overall. From which it can be concluded that the ViT can be considered for AOI applications. In addition, the ViT appeared to perform better on the combined dataset versus the datasets individually, leading to the belief that combining datasets can improve a ViT's overall performance.

Data Source/Code/Ethics Statement

1. Work on this thesis did not involve collecting data from human participants or animals.
2. The metal casted product dataset (Dabhi 2020) is freely available under the Attribution-NonCommercial-NoDerivatives 4.0 International (CC BY-NC-ND 4.0) license, which allows one to freely use the material for non-commercial purposes (if the author is correctly credited). The dataset is listed on kaggle.com.
3. The mechanical component analysis (Paladi 2020), which is found on kaggle.com was published without a license. Considering there is no license and the author has stated the dataset is free to use it is assumed that this is case.
4. The metal surface dataset (Lv et al. 2020) is publicly available on github.com and can be used for research as long as the author is properly referenced.

5. The ViT model that was used for this thesis ([Morales 2021](#)) is licensed under the Apache License 2.0. This means that the model can be used for commercial purposes and more. The only exceptions are that the code does not come with any warranties, cannot be trademarked and the author cannot be held liable for negligence.
6. Images used in this thesis, when not produced by the author, were licensed as described in point 2, 3 and 4.
7. The used data sources were publicly available on [kaggle.com](#) and [github.com](#).
8. The original owner of the data and code used in this thesis retains ownership of the data and code during and after the completion of this thesis.
9. The author of this thesis acknowledges that they do not have any legal claim to this data or code.
10. The code used in this thesis is publicly available here: [data pre-processing](#), and here: [models and methods](#).

1. Introduction

Manufacturing as we know it is going through a digital transformation, adopting new technologies and becoming more agile. All in order to adjust to ever increasing market demands ([Diez-Olivan et al. 2019](#)). One of the processes that manufacturers are transforming is the way quality inspections are being conducted. This process is being automated through the use of image classification for defect detection of products ([Abu Ebayyeh 2020](#)). This type of application is called Automated Optical Inspection (AOI), and in the current situation the preferred image classification algorithm used for this type of application is called the Convolutional Neural Network (CNN) ([Abu Ebayyeh 2020](#)). However, as of 2021 there is a new model that has shown to outperform state-of-the-art CNNs. The image classification model in question is called the Vision Transformer (ViT) ([Dosovitskiy et al. 2021](#)).

The ViT has already proven itself in the classification of defects in steel plates ([Fan 2021](#)) to classifying building materials from a combined construction dataset ([Soleymani et al. 2021](#)). However, these applications are very specific. In the current situation, there is a lack of research being conducted into if a ViT can be considered for a more general AOI application that can detect defects across several types of products. This leads us the primary research question of this thesis: "Can a Vision Transformer serve as a universal metal defect classification model."

In order to answer this question three publicly available datasets containing three different types of metal products (metal casted products, metal pistons and metal plates) have been combined into a single dataset for the ViT to be tested on.

The combined dataset used for this thesis has a significant class imbalance. It is expected that this will negatively impact the ViT's prediction results. Therefore, another research question has been introduced: "What is the best method for addressing the imbalanced data in order to ensure the highest possible accuracy and recall can be achieved with the ViT".

Another topic that will be investigated is whether or not certain minority classes can be combined. The combining of minority classes is expected to help with mitigating

the expected negative effects that are associated with the dataset's class imbalance. This topic has led to the last sub research question: "Does combining similar dataset classes improve the overall accuracy of the ViT."

The scientific relevance of this thesis is found in the fact that there is little known about the use of ViTs for classifying defects for different types of products, and limited knowledge on the use of ViTs for AOI applications. In addition, this research will hopefully further strengthen the belief that the CNN is not always the best solution for AOI applications, and that there are alternatives out there that can achieve similar, or even better, classification results. The overall results of this thesis as described in section 4 showed that the used (pre-trained) ViT outperformed the baseline CNN. Furthermore, the use of SMOTE and combining two classes resulted in the best results overall: precision: 96%, recall: 95%, f1-score: 96%, and accuracy: 95%. However, the ViT (and CNN) consistently under performed on the surface plate classes of the combined dataset. On the contrary, the ViT's performance on the metal casted parts and metal piston was perfect. Finally, testing the model on the datasets separately negatively impacted the ViT's performance. Therefore, it appears that the combining of datasets improved the ViT's overall performance.

2. Related work

2.1 Manufacturing

Over the past decades, manufacturers have achieved high levels of automation. They have gone from manual assembly to (almost) fully automated production lines (Diez-Olivan et al. 2019). However, even with these high levels of automation, these manufacturing processes are traditionally static-, hierarchical processes. When adjusting for increasingly changing market demands, these static processes are disrupted and incur costs (Diez-Olivan et al. 2019). Market demands require manufactures to increase their adaptability, increase product customization, all while remaining profitable (Diez-Olivan et al. 2019). These market demands, innovations in information and communication technology" (ICT), and government incentive have led to an ongoing digital transformation among manufacturers. The goal being the integration new technologies to help overcome these challenges.

One of the processes within manufacturing that is going through the digital transformation is the way quality inspections are conducted. Quality inspections can be divided into two methods: manual inspections (performed by humans) and AOI which is automated (Abu Ebayyeh 2020). There are different setups possible but, in essence, the AOI setup consists of an image sensor with a light source that are directed at the products that are to be inspected. The generated images are then either pre-processed first, or directly fed into an image classification model. The classification model will then classify the product images as non-defective, defective, or depending on the setup even classify the type of defect. The adoption of AOI will lead to significant productivity gains, increased accuracy of the quality inspection process, and a reduction in overall costs (Abu Ebayyeh 2020).

2.2 Image Classification models for AOI applications

In the current situation CNNs are the preferred method in imaging applications (Abu Ebayyeh 2020) and have become one of the most commonly used algorithms for inspection purposes (Cha 2017). A large number of studies have been conducted into

the CNN architecture and (fueled by ImageNet like challenges) have resulted in a wide range of state-of-the-art CNN models. One of these models that contributed to the CNN becoming the dominant model for image classification was the 2012 AlexNet model (Krizhevsky, Sutskever, and Hinton 2012). Many other image classification models such as the VGG (Simonyan and Zisserman 2015), ResNet (He et al. 2015) and DenseNet (Huang, Liu, and Weinberger 2016) followed. All improving upon their predecessors' ImageNet Performance. Many of these image classification models are freely available, making it relatively easy to utilize these state-of-the-art Image Classification Models for AOI applications (Stojnic et al. 2021).

2.3 Vision Transformer

In 2021, a novel image classification model called the ViT showed promising image classification results. In some cases, it even outperformed state-of-the-art CNNs (Dosovitskiy et al. 2021), which on itself is an achievement. However, the fact that this was achieved whilst requiring far fewer computational resources (factor of 2 to 4) for training make it stand out as an alternative to the CNN (Dosovitskiy et al. 2021). ViTs take a very different approach to image classification as their CNN counterparts. In essence, a ViT is an altered Transformer that has been modified from its Natural Language Processing (NLP) purpose to image classification (Dosovitskiy et al. 2021). The ViT is based on the self-attention based architecture (Dosovitskiy et al. 2021), which enables it to relate different sections of a single image in order to compute a representation of the whole image (Vaswani 2017). This enables the ViT to determine which image sections are most important for classification.

In 2021, researchers showed that the use of a ViT for defect classification in steel plates outperformed a state-of-the-art ResNet50 (CNN). Therefore, they concluded that the ViT has superior performance compared to other methods and stated that the ViT can be considered for AOI applications (Jiangling et al. 2021). However, their research had a few shortcomings. For one, each of their classes only contained 300 examples, and they did not pre-train the classification models. This is not an issue considering the purpose of their study, which was to compare two types of image classification models. However, when trying to achieve a higher accuracy score, it would've been beneficial to use transfer learning, which will compensate for the lack of data (Soleymani et al. 2021). Secondly, the researchers used a ResNet50 to compare their ViT to. However, based on ImageNet accuracy scores it can be concluded that there are many other image classification models that they could've used that are better than the ResNet50 e.g. Fix_ResNet50_vd_ssl, Assemble-ResNet152 (Stojnic et al. 2021).

2.4 Transfer Learning

One reoccurring issue with using these state-of-the-art CNNs, or any CNN model, is the fact that they require vast amounts of training data (Miotto et al. 2018). Since a lack of training data is a common challenge for manufacturers (Kim et al. 2017), a pre-trained version of these image classification models might be a solution for this problem. The concept behind pre-training is called "transfer learning". In short, when using transfer learning an image classification model is first trained on a sufficiently large dataset (e.g. ImageNet dataset), and then fine-tuned on the new image classification dataset of which there is not a sufficient amount of data (Gosh et al. 2018). This, in turn, reduces the amount of required training data, and computational resources that are needed for training (Abu Ebayyeh 2020). Another way of describing transfer learning is as follows:

"Transfer Learning is based on the idea that learning based on one specific problem can be re-utilized for solving another set of problems"-(cf. [Gosh et al. 2018](#), pg. 246).

In 2020, a group of researchers compared an AlexNet, VGG16, ResNet50, Densenet-121, and a MobileNetV3-Large on a welding dataset ([Yang et al. 2020](#)). The first four models were tested with and without pre-training on the test-, validation data. They used two different test setups on which they trained and tested their models. The first setup consisted of a binary classification task that classified the images either as defective product or quantified product. For the second test setup the images were classified as: Porosity, Level Misalignment and Normal ([Yang et al. 2020](#)). When classifying three classes (test setup 2) the majority of (non)pre-trained models outperformed the pre-trained models. Whereas, when two classes were used (test setup 1) the majority of the pre-trained models outperformed the (non)pre-trained models. Concluding that pre-training is not always a solution to an image classification problem, but in the right situation can improve a model's results ([Yang et al. 2020](#)).

Since the ViT is a relatively new model there is little research available on the effect of transfer learning on a ViT's performance. However, the research that showed the ViT outperforming state-of-the-art CNNs achieved this by first being pre-trained ([Dosovitskiy et al. 2021](#)). Furthermore, in their research they state that in order to achieve this high level of performance a sufficiently large dataset is required for pre-training ([Dosovitskiy et al. 2021](#)). Therefore, it can be concluded that in order to achieve an acceptable level of accuracy for an AOI application that uses a ViT, the ViT should be pre-trained.

In 2021, researches used a pre-trained ViT model to classify building materials and achieved 98% accuracy scores or higher ([Soleymani et al. 2021](#)). Their classes were imbalanced and ranged from 42 to 464 examples. These results further confirm that pre-training is advised when dealing with smaller and imbalanced datasets. In addition, in their research their ViT was pre-trained on different datasets and they came to the conclusion that their best performing ViT was pre-trained on the largest dataset ([Soleymani et al. 2021](#)). Implying that the largest available dataset should be used for pre-training a ViT if possible. Due to the size of the dataset that was used for this thesis and the fact that a ViT on average needs more training data than a CNN ([Dosovitskiy et al. 2021](#)) Transfer Learning will be used. This will help mitigate the negative effects caused by a lack of training data. This is in line with what Dosovitskiy did in his research into ViT models ([Dosovitskiy et al. 2021](#)).

2.5 Universal application

Most AOI applications are developed for a single defect detection purpose, a single product type to be classified ([Abu Ebayyeh 2020](#)). This is a shortcoming because manufacturers often produce multiple types of products, implying that they need to re-train their model for each specific product. Even though this is a limitation, little research has been conducted into this topic. To date, only a few research papers have been found. In one pre-print paper two researchers investigated how a modified VGG19 (called the Multi-path VGG19) would perform on six considerably different industrial datasets ([Apostolopoulos 2020](#)). Their results ranged from 77% accuracy up to 99% accuracy depending on the dataset. However, in their research they train and test their model per dataset individually. They do not combine the datasets, they only re-applied the same model with some tuning per dataset. Therefore, it cannot be called a general purpose image classification model.

As was briefly described in the Transfer Learning section 2.4, researchers investigated the use of a pre-trained ViT for classifying building materials (Soleymani et al. 2021). What sets them apart from the previous VGG19 example (besides using a ViT), is that they actually combined two imbalanced datasets into one. On their combined dataset they achieved a prediction accuracy of 97% or higher (Soleymani et al. 2021). Even though the results are impressive the researchers did use very similar datasets for their classification task. All were surfaces such as brick and pavement. As stated in their future work section, it would be interesting to see how well this model would generalize if their dataset contained a more varied set of classes (Soleymani et al. 2021). Furthermore, in their research they required 5-folds to achieve their results. This approach makes running their model computationally expensive compared to doing a single train-, validation and test split. It would be interesting to see how far their model would come with only a single train-, validation and test split compared to using their 5-fold approach.

This thesis is similar to the research done in (Soleymani et al. 2021). However, instead of using two very similar datasets, three very different datasets are used. In addition, instead of the computationally expensive K-fold the normal train-, validation and test split is used. The train data will then be augmented using Geometric Data Augmentation (Shorten 2019). Augmenting the training data will artificially increase the number of training examples. From a computational perspective this is a cheaper choice than using the k-fold technique. Finally, the ViT model that is used for their research was pre-trained on a larger dataset compared to this thesis's ViT, which can only be pre-trained on the ImageNet dataset (Morales 2021).

2.6 Class Imbalance

Imbalanced classes cause CNNs' to over classify the majority class and under classify the minority class. This impacts the classification model's overall performance (Johnson 2019). Even though it is assumed ViTs will display similar behavior, this has not been extensively researched. In the (Soleymani et al. 2021) example mentioned in section 2.5, their dataset was imbalanced, but the researchers did not use any re-sampling, or other methods to help address the imbalance. Still, they achieved high accuracy scores so it could be that ViTs are less susceptible to the negative effects of imbalanced classes like its CNN counterpart. However, this will need to be researched. This brings us to how to mitigate the expected negative effects of the class imbalance. These methods can be divided into two groups: Data Level Methods and Algorithm Methods (Johnson 2019).

2.6.1 Data Level Methods. Among the Data Level Methods there are (among others) Random UnderSampling (RUS), Random Oversampling (ROS), and Synthetic Minority Over-sampling Technique (SMOTE) (Johnson 2019). When applying the RUS method instances from the non-minority classes are removed until each class contains the same number of examples as the minority class (Johnson 2019). This method is beneficial when trying to divert attention from the larger classes to the minority class. However, one major drawback is the fact that instances and therefore data is removed. This can negatively impact a classification model's ability to correctly classify instances. For the purpose of this thesis, RUS is not a logical approach. This is because too much training data would be removed to make the non-minority classes match the minority class.

With ROS minority class entries are randomly copied to artificially increase the non-majority classes until they are the same size as the majority class (Masko and Hensman 2015). In Masko and Hensman's research they concluded that the ROS technique results

in statistically significant improvements when it came to the results of their classification task (Masko and Hensman 2015). Oversampling is seen as a robust approach for mitigating the negative effects of imbalanced classes. However, ROS does come a drawback. Primarily that ROS can lead to an image classification model overfitting, which will negatively impact the model's overall performance (Buda, Maki, and Mazurowski 2018).

To help mitigate the negative effects of the ROS method, a new method was developed called SMOTE. SMOTE takes a different approach by taking a minority class example and by combining it with the minority classes' k-nearest neighbors into a new artificial example (Chawla et al. 2002). The benefit being that the classification model being trained on the artificial examples is less likely to over-fit compared to when ROS is used (Buda, Maki, and Mazurowski 2018).

2.6.2 Algorithm Level Methods. Unlike with the Data Level Methods that modify the data being fed into the model, the Algorithm Level Methods help the classification models adjust for the imbalance found in the data. There are several methods that can be applied, but the choice for this thesis was to use Cost Sensitive Learning.

When using Cost Sensitive Learning the model assigns different costs to each class depending on the size of the class. Minority classes will be assigned more weight compared to majority classes. A higher cost of a class means that incorrectly classifying an instance from the class carries a higher penalty, therefore increasing its importance. Using Cost Sensitive Learning changes the purpose of the model from that of minimizing the total error to minimizing the total cost (Buda, Maki, and Mazurowski 2018).

3. Methods

3.1 Data

In order to test how capable a ViT is on classifying defects for different types of products three publicly available datasets have been combined into one. In the following subsections the datasets are explained.

3.1.1 Casting Product Image Dataset for Quality Inspection. This dataset contains images of casted metal parts. The images are 512x512 pixels sized, grey-scaled and the jpeg format. The defective class contains 718 images and the non-defective class contains 519 images. The total size of these images combined is 30.9 MB. Shown in figure 1 is an example showing a non-defective and a defective casted metal part from this dataset. This dataset is available on kaggle.com (Dabhi 2020).



Figure 1
Examples of casting product image dataset images(non-defective, defective) (Dabhi 2020).

3.1.2 Defects location for metal surface. The following dataset contains 3570 grey-scaled, jpg formatted images, that are 2048x1000 sized. The dataset consists of 10 different defect classes: Punching hole, Welding line, Crescent gap, Water spot, Oil spot, Inclusion, Rolled pit, Folding, Silk spot, Crease. The classes of this dataset are imbalanced, the smallest class contains 31 files and the largest 650. Shown in figure 2 is an example of two images from the dataset. The total size of the dataset is 968 MB. This dataset is available on github.com (Lv et al. 2020).

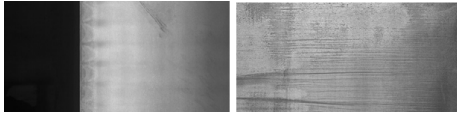


Figure 2
Metal surface image examples (Surface water spot and waist folding) (Lv et al. 2020).

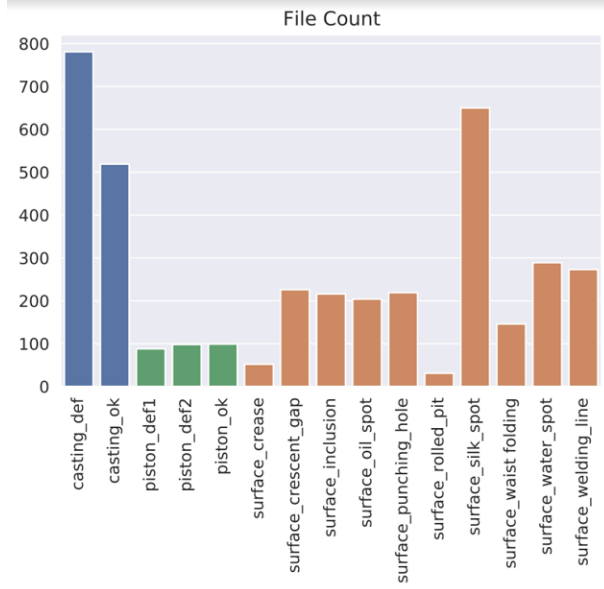
3.1.3 Mechanical Component Images. The last dataset that will be explored is the Mechanical Component images dataset. It consists of three classes of images that are 90x90 pixels, rgb colored, png formatted. The dataset contains images of pistons used in air conditioning units. The first class is called defect1 and contains the: broken-, incorrect shape- and fallen piston images. The second class is called defect2 and contains the: oil-, grease- and rust stains piston images. The third class is the non-defective class that consists of non-defective piston images. The defect1 class consists of 88 images, the defect2 class consists of 98 images and the non-defective class consists of 99 images. In total there are 285 images. The total size of the images combined is 4.83 MB and the images are accessible on kaggle.com (Paladi 2020). Shown in figure 3 is an example of two images from the Mechanical Component dataset.



Figure 3
Examples of mechanical component images (non-defective, defective from defect2) (Paladi 2020).

3.1.4 Combined dataset overview. The combined datasets amount to a total of 3891 examples distributed over 15 defect and non-defect classes. Shown in figure 4 is an overview of the distribution of images across the different classes.

As shown in figure 4, the combined datasets are significantly imbalanced. The piston categories contain considerably fewer examples per class than the cast dataset, and the surface dataset has significant class size differences between its own classes as well. The smallest class in the dataset contains 31 examples, whereas the largest class contains 650 examples.

**Figure 4**

Number of images per class for all three datasets (casting, piston and metal surface)

3.2 Image processing

3.2.1 Image Pre-processing. The images saved in the data folder are opened one-by-one as an rgb image. This is required for the ViT because it has been pre-trained on colored images (Russakovsky et al. 2014). The opened image is then reshaped to the desired dimensions (224,224,3) and converted into a numpy array. The pixel values within the array range from 0 to 255. However, this range will cause unnecessarily high error rates and will require additional training time for the image classification model (Sola and Sevilla 1997). To overcome these challenges the image array needs to be re-scaled. For this thesis the choice was made to use per image standardization according to the following formula:

$$\text{standardized array} = \frac{x - \bar{x}}{\sigma_x} \quad (1)$$

As is shown in formula 1, the following is happening: The mean($\bar{x}$) of the array is subtracted from the array(x), the result is then divided by the standard deviation of the array(σ_x) resulting in the standardized array.

In addition to standardizing, normalizing the data was also tested but the image classification models under performed on the normalized data versus the standardized data. After all the image arrays are standardized they are combined into one data array with the following dimensions: (3891, 244, 244, 3) and is 4.6 GB large.

The corresponding labels are derived from directory from which the images were extracted. Using a dictionary the category name is replaced by a class specific integer. This is then used to create the labels. The labels are then combined into a numpy array with the following dimensions: (3891,). Considering that the labels are nominal (unordered) the choice was made to one-hot-encode the labels before they are

fed into the augmenter. The one-hot encoding is achieved through the use of the `tf.keras.utils.to_categorical` package that converts the labels from integers to one-hot-encoded labels. The result is a new array with the following dimensions: (3891,15).

The choice for the images to be resized to 224 by 224 was based on the research of (Dosovitskiy et al. 2021). A larger image size could potentially improve the ViT's performance, but it would also cost more memory to run. Therefore, the image size choice was made for 224 by 224.

3.2.2 Splitting the data. In order to optimize and later test the models and other techniques, the data is split into train (70%), validation (15%), and test (15%). To ensure there would be no data leakage, the test data is split from the dataset first. Only after is the validation data split from the train data (the split ratio is adjusted to ensure that the validation and test datasets would be approximately the same size).

To ensure that each subset of data is representative of the complete dataset, stratified split was used for splitting the dataset. The result is that each subset of data approximately has the same ratio of instances per class, and contain the same number of classes as the complete dataset. During the thesis, the models and other techniques were optimized using only the train and validation subsets. Only after the optimal values for the hyper parameters were found, would the model be trained on the train- and validation sets combined to predict the test dataset.

3.2.3 Data augmentation. To increase the number of training examples and to help mitigate the lack training data for some classes, Geometric Data Augmentation (Shorten 2019) was utilized for the research. Five different augmentation strategies were tested to determine which would result in the highest validation accuracy score.

1. No Augmentation
2. Vertical- and horizontal flip
3. Vertical flip, horizontal flip and 40 degree rotation
4. Vertical flip, horizontal flip, 40 degree rotation, width- and height range 0.3
5. Vertical flip, horizontal flip, 40 degree rotation, width- and height range 0.3, and zoom 0.3

The best result came from the second strategy, which consisted of flipping the images vertically and horizontally. In addition to the chosen augmentation Mixup (Zhang et al. 2017) was also tested in combination with the vertical- and horizontal flip. However, it did not result in better performance of the models. The Mixup augmentation strategy (as the names indicates) mixes features of images with those of other images (it also combines their corresponding labels). As a consequence, it ensures the model does not become overly confident about the relationship between certain features, and their corresponding labels. The benefit of this approach is that it results in a more robust image classification model (Zhang et al. 2017). However, it did not improve the models' performance, so the initial 2st augmentation strategy was chosen. Furthermore, the augmenting of images was only performed on the train data and fed into the image classification models in batches of 32.

3.3 Class Imbalance

To mitigate the negative effects of the class imbalance Random Oversampling and SMOTE were tested. These functions are readily available through the Imbalanced Learn Library (Lemaître, Nogueira, and Aridas 2017). Due to the fact that the re-sampling methods can only be performed on 2D arrays the data had to be reshaped from a 4 dimensional numpy array to a 2 dimensional array. It then must be re-sampled and reshaped back to a 4 dimensional array. At this point, it can be fed into the image augementer.

3.3.1 Cost Sensitive Learning. One of the methods that was tested was using Cost Sensitive Learning for the image classification models. Cost Sensitive Learning re-weights the classes resulting in a relatively high cost for minority classes and a relatively low cost for majority classes. This ensures that the model will put more emphasis on the minority classes over the majority classes (King and Zeng 2001). In order to perform this action, the Scikit-learn library was utilized which contains a function called `compute_class_weight` (Pedregosa et al. 2011). The function creates a weight library, which is then used by the classification model for determining the cost for each class. As an example: if there are 2 classes and class 1 contains 50 examples and class 2 contains 100 examples then the cost of class 1 will be twice as high as that of class 2, therefore adjusting for the imbalance.

3.4 Minority classes

As has been stated in the combined dataset section 3.1.4, there is a severe class imbalance. When testing the ViT model, it became clear that it under performed on the minority classes: `surface_crease` and `surface_rolled_pit`. This negatively impacted the ViT's overall performance. The `surface_crease` contains 52 examples and the `surface_rolled_pit` 31 examples. To help mitigate the negative effect of the minority classes, it was investigated if either one, or both of the minority classes could potentially be combined with other classes.

3.4.1 `surface_crease`. The best way of describing what a surface crease is, is as follows:

"A crease is a vertical transverse fold, with regular or irregular spacing across the strip, or at the edge of the strip. The main reason is the local yield along the moving direction of the strip in the uncoiling process." - (cf. Lv et al. 2020, pg. 8)

When visually inspecting the surface crease class images it became clear that the class did not share enough of a similarity between the other classes. This meant that this class could not be combined with other classes. To help mitigate the lack of instances for this class, additional datasets were investigated. The goal being that another dataset could potentially be used to increase the number of images in the surface crease class. Even though there are other datasets such as the NEU-DET (Lv et al. 2020), the found classes appeared to be too different from the surface crease to be combined. Therefore, this class cannot be improved by either combining or adding additional data at this moment.

3.4.2 `surface_rolled_pit`. When reviewing the `surface_rolled_pit` class it became apparent that this class and the `surface_inclusion` class seem to both describe similar types of metal inclusions.

1. "Rolled pits are bulges, pits on the metal surface that are punctate, flaky, strip like"- (cf. [Lv et al. 2020](#), pg. 8).
2. "Inclusion is a typical defect of metal surface defects, usually showing small spots, fish scale shape, strip shape, block irregular distribution in the strip of the upper and lower surface (global or local), and is often accompanied by rough pockmarked surfaces."- (cf. [Lv et al. 2020](#), pg. 8).

Therefore, it was decided to visually inspect both classes and it became clear that the classes look similar, and in some instances the classes even overlapped. One such example is shown in figure: 5.

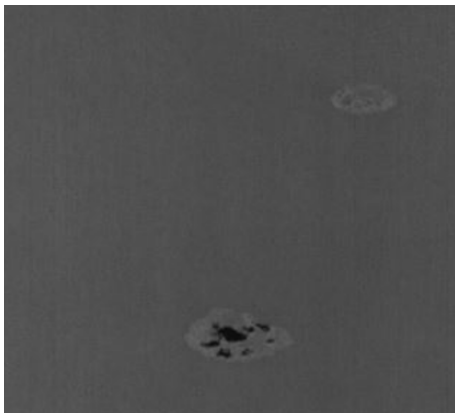


Figure 5
Examples of class overlapping image (enlarged image) ([Lv et al. 2020](#))

Based on the similarities it was decided to combine the `surface_rolled_pit` class (containing 31 images), with the `surface_inclusion` class (containing 216 images), and keeping the `surface_inclusion` class's name. The combined classes contain 247 images.

3.5 Models

3.5.1 Vision Transformer. The Vision Transformer is based on the transformer architecture and has shown promising results with image classification tasks ([Dosovitskiy et al. 2021](#)). The ViT model utilizes the attention-based transformer architecture and uses it for image classification tasks. The transformer architecture traces its roots from the field of Natural Language Processing. Within this field, it has become the model of choice for such tasks. In order for the researchers to use the transformer architecture for image classification they had to make a few modifications. The easiest way to explain the changes to the transformer is by going through the steps involved with classifying an image. An image which is to be classified is first divided into 16 equally sized patches. Each patch is then flattened and linearly embedded. The flattening is required because transformers can only take in 1 dimensional vectors. In order to retain positional information a positional embedding is added to each embedding. From here, the resulting sequence is fed into a standard transformer Encoder ([Dosovitskiy et al. 2021](#)). Finally, a classification head is applied at the end of the model to obtain the model's predictions ([Soleymani et al. 2021](#)). Shown on the left side of figure 6 is the ViT architecture that was just described.

The encoder used in the transformer architecture consists alternating layers of multi-head (self)-attention layer and multi-layer perceptron (MLP) blocks. Each block starts with a normalization layer, followed by either a multi-head (self)-attention layer, or a MLP layer (depending on the block). The ViT base that was used for this thesis consisted of 12 layers (each containing both blocks). Within the multi-head attention layer, a weighted sum for all values in the input sequence is calculated. The weights are based on the pairwise similarity between all possible combinations within the input vector (Dosovitskiy et al. 2021). This allows the model to determine what to attend to within the input sequence.

The model used for this thesis came from github.com (Morales 2021) and uses a softmax layer at the end of the model for the final classification. In addition, the model is pre-trained on ImageNet and includes its top layer, which is not pre-trained. Even though the model performed well, it could not be pre-trained on other datasets which was a drawback considering research (Soleymani et al. 2021) has shown that a larger dataset for pre-training would've improved the ViT's performance. When choosing the available ViT models, the ViT base and ViT large were tested with 16 and 32 patches. The ViT large ended up being too big and caused the system to run out of memory. Therefore, the choice was made for the ViT base version. Then the ViT base was chosen with 16 patches, because this resulted in the best overall performance. Furthermore, considering the model was ready from the start, there were few parameters that could be optimized. After re-running the model several times the learning rate of 1-e4 was found to be optimal and the label smoothing for the Categorical Cross entropy achieved the best results with a value of 0.2. The ViT parameters were obtained by training on the training data and testing on the validation data.

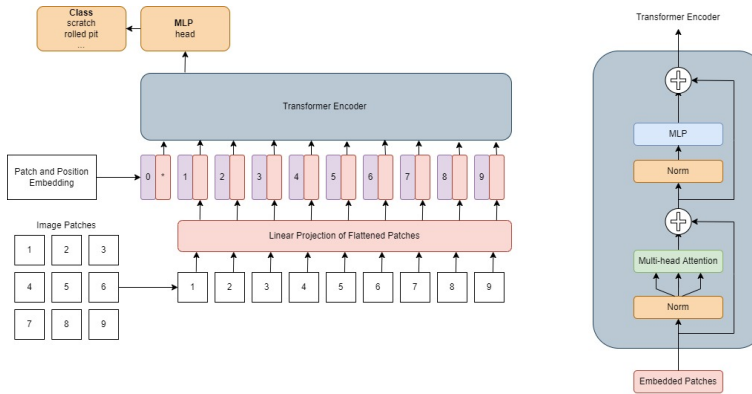


Figure 6
Example of the ViT architecture, left side is ViT architecture and right side is an example of the Transformer Encoder (Dosovitskiy et al. 2021).

3.5.2 Convolutional Neural Network (Baseline). The baseline model that is used to compare the ViT with is a CNN. The CNN model is based on an adaptation of the LeNet-5 architecture (Buda, Maki, and Mazurowski 2018). The model base consisted of 2 convolutional layers, 2 max pooling layers, 1 flatten layer, and 2 fully connected layers. However, 2 additional convolutional- and 2 max pooling layers were added to increase validation accuracy. One of the primary benefits using LeNet-5 adaptation versus the classical LeNet-5 architectures is the fact that this model makes use of max-pooling layers (Buda, Maki, and Mazurowski 2018). The max pooling layers down sample the

image to reduce the complexity for the upcoming layers (Albawi et al. 2017). Adding max pooling layers to the model significantly reduces the computational resources required for training the model. For the convolutional layers and fully connected layers rectified linear unit (ReLU) activation was used (Zeiler et al. 2013). Using ReLu in deep learning results in the faster convergence, is faster to compute and generalizes better (Zeiler et al. 2013). Since the model was originally overfitting on the validation data a dropout layer was added. After several runs the optimal dropout value was found to be 0.3 (Baldi and Sadowski 2013). The CNN baseline model used for this thesis is shown in figure 7. Furthermore, the parameters and number of layers used for the CNN were determined by training the model on the training data and testing them on the validation data.

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 224, 224, 32)	896
max_pooling2d (MaxPooling2D)	(None, 112, 112, 32)	0
conv2d_1 (Conv2D)	(None, 112, 112, 64)	18496
max_pooling2d_1 (MaxPooling2D)	(None, 56, 56, 64)	0
conv2d_2 (Conv2D)	(None, 56, 56, 64)	36928
max_pooling2d_2 (MaxPooling2D)	(None, 28, 28, 64)	0
conv2d_3 (Conv2D)	(None, 28, 28, 128)	73856
max_pooling2d_3 (MaxPooling2D)	(None, 14, 14, 128)	0
flatten (Flatten)	(None, 25088)	0
dense (Dense)	(None, 32)	802848
dropout (Dropout)	(None, 32)	0
dense_1 (Dense)	(None, 15)	495
Total params: 933,519		
Trainable params: 933,519		
Non-trainable params: 0		

Figure 7
CNN model overview

3.6 Training

For the purpose of the thesis a ViT base was used. The ViT base contained 12 layers, used 16 patches and was pre-trained on the ImageNet dataset containing 1+ million images. The model had 50 epochs for training but because early stopping was used the model generally did not go past 30 epochs. The used learning rate was $1e-4$ and the Rectified Adam optimization function was used (Liu et al. 2019), batch size was 32 and the Categorical Cross entropy loss function was used with 0.2 label smoothing. These choices were made based on the model's performance with predicting the validation data. For training a RTX A4000 GPU was utilized with 45 GB of memory. The processing power was provided by paperspace.com.

3.7 Performance Metrics

For the thesis, the four standard performance metrics were used for evaluating model performance. The metrics are: precision, recall, f1-score and accuracy. In order to calculate these metrics the following elements need to be known:

- True Positive (TP): Correctly Predicts Positive Class
- False Positive (FP): Incorrectly Predicts Positive Class
- True Negative (TN): Correctly Predicts Negative Class
- False Negative (FN): Incorrectly Predicts Negative Class

The elements are then used to calculate the performance metrics:

- Precision = $TP / (TP + FP)$
- Recall = $TP / (TP + FN)$
- F1-score = $(2 * Precision * Recall) / (Precision + Recall)$
- Accuracy = $(TP + TN) / (TP + TN + FP + FN)$

Regarding the metrics themselves, all are important. However, taken into consideration that the focus lies with AOI applications the most important metric is recall (probability that an actual defect is detected). The reason being that defects should not be missed, because they can cause issues further along in the manufacturing process, or worse when the customer receives a defective product. Even though Recall is important, the model should also not cause too many false alarms (FP). If one gets too many false alarms one cannot be sure the product is actually defective. Therefore, an inspection is required to verify if the product is actually defective, this can be a time consuming task. This on itself is better than missing a defect altogether, but should still be kept to a minimum. That's why the second metric that is important is precision (probability a classified defect product is actually defective).

4. Results

In this section, the image classification results of the ViT and CNN on the combined dataset 3.1 will be described. The results of the experiments are shown in table 1. As is shown in the table the baseline CNN and ViT including the tested approaches are listed. The ViT model was tested with and without being pre-trained. This was done in order to confirm that the choice for pre-training the model was the correct one.

Regarding the methods addressing the class imbalance, the oversampling techniques (RandomOverSampling and SMOTE) both improved the baseline ViT's classification performance. However, as research indicated (Buda, Maki, and Mazurowski 2018), and was expected the SMOTE method resulted in the best results of the two. On the contrary to other research (López et al. 2012), the Cost Sensitive Learning approach actually resulted in the ViT performing worse than the ViT baseline. From these results it can be derived that the Cost Sensitive Learning approach did not reduce the negative effects caused by the class imbalance. Furthermore, the combining of the surface_rolled_pit class and the surface_inclusion showed to further improve the ViT's classification performance. The best ViT performance was achieved by using SMOTE in conjunction with merging the surface_rolled_pit and the surface_inclusion classes.

In addition to the overall performance shown in table 1, there is a discrepancy between the models' performance per dataset. With the ViT using SMOTE and combining the classes, perfect performance is achieved on the mechanical component dataset 3.1.3, and the casting dataset 3.1.1. However, this is not the case for the surface dataset 3.1.2. As is shown in table 2, it appears to be a harder dataset to classify. This discrepancy

Table 1
Performance of all methods and models

Model	Method	<i>Results</i>			
		Precision	Recall	F1	Accuracy
ViT	Baseline (without pre-training)	0.73	0.42	0.51	0.42
	Baseline	0.98	0.84	0.89	0.84
	RandomOverSampling	0.94	0.90	0.92	0.90
	SMOTE	0.96	0.92	0.94	0.92
	Cost Sensitive Learning	0.99	0.62	0.69	0.62
	Combined Classes	0.99	0.90	0.93	0.90
	Combined Classes and SMOTE	0.96	0.95	0.96	0.95
CNN	Baseline	0.93	0.65	0.73	0.65

Note. Due to the class imbalance, weighted averages are reported for: Precision, Recall and F1 score. The boldface entries achieved the best performance on that metric.

between model performance per dataset was seen with all models and all methods. When an attempt was made to run the ViT model, SMOTE and combining classes on the surface data only it performed worse (accuracy: 24%), compared to when the dataset included the casting and piston datasets (accuracy: 0.95%).

Table 2
ViT performance per class(Combined classes & SMOTE)

	Name	<i>Results</i>			
		Precision	Recall	F1	Support
Classes	casting_def	1.00	1.00	1.00	117
	casting_ok	1.00	1.00	1.00	78
	piston_def1	1.00	1.00	1.00	13
	piston_def2	1.00	1.00	1.00	15
	piston_ok	1.00	1.00	1.00	15
	surface_crease	0.83	0.62	0.71	8
	surface_crescent_gap	1.00	0.91	0.95	34
	surface_inclusion	0.94	0.86	0.90	37
	surface_oil_spot	0.91	0.94	0.92	31
	surface_punching_hole	0.93	0.82	0.87	33
	surface_welding_line	0.86	0.90	0.88	41
	surface_silk_spot	0.95	0.99	0.97	97
	surface_waist_folding	0.95	0.95	0.95	55
	surface_water_spot	0.97	0.91	0.94	43
Averages	micro avg	0.96	0.95	0.96	584
	macro avg	0.95	0.92	0.95	584
	weighted avg	0.96	0.95	0.96	584
	samples avg	0.95	0.95	0.95	584

5. Discussion

The goal of this thesis was to determine if a ViT could be considered as an alternative to the CNN for AOI applications, where different types of metal products are inspected. This goal was phrased as the main research question: "Can a pre-trained ViT serve as a universal metal defect classification model". To properly investigate this topic, the ViT was tested on three different types of metal products from a combined dataset. The combined dataset contains images of pistons, casted products, and metal surfaces. In order to determine if the ViT is a suitable model for this task, a pre-trained ViT model was compared to a CNN model. In addition, several methods were tested in order to mitigate the negative effects of the imbalanced dataset. The results of the models and used methods can be found in section 1.

5.1 Vision Transformer

The first experiment that was conducted was to determine how well the ViT would perform compared to the CNN baseline. As is shown in the results of section 1, the CNN achieved a higher precision, recall, f1 score, and accuracy when compared to the (non)pre-trained ViT. This result further confirms that a ViT generally need more training data than its CNN counterpart (Dosovitskiy et al. 2021). However, when compared to the pre-trained ViT the CNN under performs by a large margin. The ViT achieved better results across all performance metrics. From these results one can derive that the ViT is the better choice when compared to a standard CNN model. This is supported by other studies described in section 2.3.

5.2 Imbalance

The second experiment was focused on answering the following sub research question: "What is the best method for addressing the imbalanced data to ensure the highest possible accuracy and recall can be achieved with the pre-trained ViT". In order to answer this question, 2 data- and 1 algorithm level method were investigated. In line with literature described in the results section 4, using the SMOTE method resulted in the best performance increase of the ViT model. This was expected because the SMOTE method was developed to improve upon the overfitting nature of ROS (Buda, Maki, and Mazurowski 2018). The Cost Sensitive Learning method ended up resulting in worse performance than the ViT baseline, contradicting known research (López et al. 2012).

5.3 Class Combinations

The third experiment was conducted to determine if certain minority classes could be combined. The goal being that smaller, hard to classify classes could be combined with other classes, therefore improving the overall performance of the ViT. This was summarized in the following sub research question: "Does combining similar dataset classes improve the overall accuracy of the pre-trained ViT." Only two classes ended up being similar enough to be combined. The classes are: `surface_rolled_pit` and `surface_inclusion`. In theory, the piston defect 1 and defect 2 could've been combined as well. However, the ViT already achieved 100% accuracy with predicting these two classes, therefore combining them was not required for improving the ViT's performance.

5.4 Performance across datasets

One interesting result that was discovered was that all the models- and methods' performance appeared to be worst for the surface dataset. Based on the overall results, one might conclude that the dataset is harder to classify than the other two datasets. However, based on the results of table 2 can be derived that among the surface dataset's classes there appears to be a correlation between the achieved class accuracy and number of examples. Because, stratified split was used for splitting the data the class distribution is about the same as that of the original dataset. Implying that the smaller classes in test set also have fewer training instances for the model to train on, therefore negatively impacting the models' performance for those classes.

On the contrary to the surface dataset, the ViT achieves excellent results classifying the piston dataset. It is believed that this is because unlike the other two datasets (which originally were grey-scaled), the piston dataset was originally rgb colored. Therefore, the ViT had more information to train on. In addition, because the ViT is pre-trained on colored images it will probably perform better on images that are rgb colored from the start, instead of grey-scaled images that have been converted to rgb color.

Another interesting observation was made when the ViT was tested on just the surface dataset. The ViT performed considerably worse (accuracy: 24%) compared to when it was tested on the combined dataset (accuracy: 95%). From these results it appears that the ViT's performance improved by having more images to train on, even if the type of products in the images are very different. This behavior appears to be similar to the concept of transfer learning which was described in the literature section 2.4 where a model is trained on a generic dataset in order to improve the accuracy of the model on another dataset. However, in this case the ViT learns from the piston and casting dataset, in order to improve the results of the surface dataset.

6. Conclusion

The aim of this thesis was to answer the following research question: "Can a pre-trained ViT serve as a universal metal defect classification model." The results (shown in table 1) show that the pre-trained ViT outperformed the baseline CNN on this task. Furthermore, several methods were tested to determine which approach would be best suited for mitigating the negative effects of the dataset's class imbalance. ROS, SMOTE, and Cost Sensitive Learning were compared and the SMOTE approach achieved the best results across 3 out of 4 metrics. Therefore, it was concluded to be the best option. In addition to utilizing SMOTE, two of the dataset's classes showed to be similar and certain instances even overlapped. Therefore, it was decided that these two classes could be combined. The merging of these classes, combined with using SMOTE resulted in the best ViT results overall. Based on the results described in table 1, the ViT achieves good results on average and depending on the application can be considered for AOI applications. Regarding the ViT's performance, perfect results were achieved on the casting and piston datasets. However, the model under performed on the surface dataset.

This thesis is the first study investigating the use of a ViT model on three very different industrial datasets. The findings suggest that ViT models can be considered as an alternative to the CNN for AOI applications. In addition, having more types of products and therefore more data, improved the model's overall performance. This observation would suggest that developing a multi-purposed ViT based AOI application could be considered for not only practical reasons (not having to re-train the model for each product group), but also to improve the model's overall performance.

Further research should be conducted into if other augmentation techniques could further improve upon the current results. In addition, as this model was limited in its pre-training options it would be interesting to see if, and by how much, the ViT's performance improves by using a larger dataset for pre-training. Furthermore, considering that the combined dataset only contained 3 types of products, future research should be conducted into using a ViT for defect detection in a larger dataset of more varied products. Finally, as the ViT has shown to be a good alternative to the CNN, additional research will need to be done into how the ViT can be adopted as an AOI application.

References

- Abu Ebayyeh, A. R. 2020. A review and analysis of automatic optical inspection and quality monitoring methods in electronics industry. *IEEE*, 11:83192–18327.
- Albawi, S. T. et al. 2017. Understanding of a convolutional neural network. In *International Conference on Engineering and Technology (ICET)*, pages 1–6, IEEE, Antalya, Turkey.
- Apostolopoulos, D. L. 2020. Industrial object, machine part and defect recognition towards fully automated industrial monitoring employing deep learning, the case of multilevel vgg19. *arXiv e-prints*.
- Baldi, Pierre and Peter J Sadowski. 2013. Understanding dropout. *Advances in neural information processing systems*, 26:2814–2822.
- Buda, Mateusz, Atsuto Maki, and Maciej A. Mazurowski. 2018. A systematic study of the class imbalance problem in convolutional neural networks. *Neural Networks*, 106:249–259.
- Cha, Y. J. 2017. Deep learning-based crack damage detection using convolutional neural networks. *Computer-Aided Civil and Infrastructure Engineering*, 32, issue 5:361–378.
- Chawla, N. V., K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer. 2002. Smote: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16:321–357.
- Dabhi, R. 2020. casting product image data for quality inspection.
- Diez-Oliván, Alberto, Javier Del Ser, Diego Galar, and Basilio Sierra. 2019. Data fusion and machine learning for industrial prognosis: Trends and perspectives towards industry 4.0. *Information Fusion*, 50:92–111.
- Dosovitskiy, A., L. Beyer, A. Kolesnikov, et al. 2021. An image is worth 16x16 words: Transformers for image recognition at scale.
- Fan, J. X. 2021. Detection of surface defects of steel plate based on vit. *Journal of Physics: Conference Series*, 2002 012039.
- Gosh, B. P., P. Sasmal, P. Gadde, M.K. Bhuyan, and Y. Iwahori. 2018. Defect classification of printed circuit boards based on transfer learning. In *Applied Signal Processing Conference (ASPCON)*, IEEE, Kolkata India.
- He, Kaiming, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2015. Deep residual learning for image recognition. *CoRR*, abs/1512.03385.
- Huang, Gao, Zhuang Liu, and Kilian Q. Weinberger. 2016. Densely connected convolutional networks. *CoRR*, abs/1608.06993.
- Jiangling, F et al. 2021. Detection of surface defects of steel plate based on ViT. 2002(1):012039.
- Johnson, Khoshgoftaar T.M., J.M. 2019. Survey on deep learning with class imbalance. *Journal of Big Data*, 6:164.
- Kim, S. W. et al. 2017. Transfer learning for automated optical inspection. In *International Joint Conference on Neural Networks*, pages 2517–2524, IEEE, Anchorage, AK, USA.
- King, Gary and Langche Zeng. 2001. Logistic regression in rare events data. *Political Analysis*, 9:137–163.
- Krizhevsky, Alex, Ilya Sutskever, and Geoffrey E Hinton. 2012. Imagenet classification with deep convolutional neural networks. In *Advances in Neural Information Processing Systems*, volume 25, Curran Associates, Inc.
- Lemaître, Guillaume, Fernando Nogueira, and Christos K. Aridas. 2017. Imbalanced-learn: A python toolbox to tackle the curse of imbalanced datasets in machine learning. *Journal of Machine Learning Research*, 18(17):1–5.
- Liu, Liyuan, Haoming Jiang, Pengcheng He, Weizhu Chen, Xiaodong Liu, Jianfeng Gao, and Jiawei Han. 2019. On the variance of the adaptive learning rate and beyond. *CoRR*, abs/1908.03265.
- Lv, Xiaoming, Fajie Duan, Jia-jia Jiang, Xiao Fu, and Lin Gan. 2020. Deep metallic surface defect detection: The new benchmark and detection network. *Sensors*, 20(6).
- López, Victoria, Alberto Fernández, Jose G. Moreno-Torres, and Francisco Herrera. 2012. Analysis of preprocessing vs. cost-sensitive learning for imbalanced classification. open problems on intrinsic data characteristics. *Expert Systems with Applications*, 39(7):6585–6608.
- Masko, David and Paulina Hensman. 2015. The impact of imbalanced training data for convolutional neural networks.
- Miotto, R. F., F. Wang, S. Wang, X. Jiang, and J. T. Dudley. 2018. Deep learning for healthcare: review, opportunities and challenges. *Brief Bioinform*, 19:1236–1246.
- Morales, Fausto. 2021. Vit-keras. Github.
- Paladi, S. 2020. In *Mechanic Component Images (Normal / Defected)*, kaggle.com.

- Pedregosa, F. V. et al. 2011. Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, 12:2825 – 2830.
- Russakovsky, Olga, Jia Deng, Hao Su, Jonathan Krause, Sanjeev Satheesh, Sean Ma, Zhiheng Huang, Andrej Karpathy, Aditya Khosla, Michael S. Bernstein, Alexander C. Berg, and Li Fei-Fei. 2014. Imagenet large scale visual recognition challenge. *CoRR*, abs/1409.0575.
- Shorten, Khoshgoftaar-T.M., C. 2019. A survey on image data augmentation for deep learning. *Big Data*, 6:60.
- Simonyan, Karen and Andrew Zisserman. 2015. Very deep convolutional networks for large-scale image recognition. In *International Conference on Learning Representations*.
- Sola, J. and J. Sevilla. 1997. Importance of input data normalization for the application of neural networks to complex industrial problems. *IEEE Transactions on Nuclear Science*, 44(3):1464–1468.
- Soleymani, Maryam, Mahdi Bonyani, Hadi Mahami, and Farnad Nasirzadeh. 2021. Construction material classification on imbalanced datasets for construction monitoring automation using vision transformer (vit) architecture. *CoRR*, abs/2108.09527.
- Stojnic, Robert, Marcin Kardas, Viktor Kerkez, Guillem Cucurull, and Ludovic Viaud. 2021. Papers with code - the latest in machine learning.
- Vaswani, A. N. 2017. Attention is all you need. *arXiv.org*.
- Yang, Yatao, Longhui Pan, Junxian Ma, Runze Yang, Yishuang Zhu, Yanzhao Yang, and Li Zhang. 2020. A high-performance deep learning algorithm for the automated optical inspection of laser welding. *Applied Sciences*, 10(3).
- Zeiler, M.D., M. Ranzato, R. Monga, M. Mao, K. Yang, Q.V. Le, P. Nguyen, A. Senior, V. Vanhoucke, J. Dean, and G.E. Hinton. 2013. On rectified linear units for speech processing. In *2013 IEEE International Conference on Acoustics, Speech and Signal Processing*, pages 3517–3521.
- Zhang, Hongyi, Moustapha Cissé, Yann N. Dauphin, and David Lopez-Paz. 2017. mixup: Beyond empirical risk minimization. *CoRR*, abs/1710.09412.